ChatGPT

Part 3 代码落地计划书：SAM2 掩码生成 + ProPainter 时序补充 + Diffusion 增强

Executive summary

本计划书面向课程 Project 3 的 Part 3“探索优化与扩展”要求：在复现/集成 SOTA 模块的基础上识别局限并做改进。其中课程已明确提出：Part 2 可选用 SAM2 生成高质量动态物体 mask，并用 ProPainter 做视频时序修复；Part 3 的一个推荐方向是“引入 Diffusion”，因为当背景在整段视频中从未出现时，ProPainter 的传播机制无法“借像素”，可考虑用 Stable Diffusion（含 Inpainting/ControlNet）修复关键帧、再向其他帧传播。filecite turn4file1 filecite turn4file3

基于该思路，本方案把 ProPainter 作为“时序补充主干”，Diffusion 作为“补洞增强/兜底”，并将 SAM2 作为统一的 mask 来源：**SAM2 → ProPainter（完成全序列时序补全）→ Diffusion（按 hard region/关键帧做局部增强）→ 传播/融合 → 输出视频**。其中关键工程点是：在不大改 ProPainter 代码的前提下，增加“Diffusion 只对少量关键帧、少量区域执行”的策略，以控制算力开销并降低逐帧 Diffusion 导致的闪烁（flicker）。方案同时对课程交付物做对齐：公开 GitHub [1] 仓库、可复现实验脚本、必选数据集处理视频打包等。filecite turn4file4

假设与未指定项

下表列出**你已明确**与**尚未给出**（标注“未指定”）的前提；对未指定项给出可落地默认值，避免计划无法执行。

类别	项	状态	默认/建议值（可改）	依据/说明
目标路线	ProPainter 为主、Diffusion 为补充	已指定	按“关键帧/硬区域”局部 Diffusion；其余完全依赖 ProPainter	与课程 Direction C“关键帧 Diffusion + 传播”一致 filecite turn4file3，也与上文既定思路一致 filecite turn3file0
代码语言	Python	未指定	Python 3.10+	SAM2 官方要求 `python>=3.10` [2]
深度学习框架	PyTorch	未指定	PyTorch 2.5.1+（随 SAM2），并验证 ProPainter 兼容性	SAM2 官方要求 `torch>=2.5.1` [2]；ProPainter README 给出较老下限，需做兼容验证与风险预案 [3]
运行环境	OS/驱动/CUDA	未指定	Linux（Ubuntu 22.04）+ CUDA 12.x；Windows 建议 WSL	SAM2 安装建议 Windows 用 WSL，且编译 CUDA 扩展需 nvcc 与 CUDA Toolkit [2]

类别	项	状态	默认/建议值（可改）	依据/说明
硬件	GPU/显存/CPU	未指定	1×NVIDIA [4] GPU（≥12GB VRAM 起步；更推荐 ≥24GB）+ 16C CPU + 64GB RAM	ProPainter 在不同分辨率/子视频长度下显存需求差异很大（例如 720×480、80 帧、fp16 约 8GB；1280×720、80 帧可能 OOM/约 25GB） [5]
数据集	使用哪些数据/是否有 GT	未指定	必做：Wild Video + bmx-trees + tennis；可选：DAVIS（带 GT）	课程要求必做数据并提交处理视频；DAVIS 推荐用于高分且带 GT 可做 PSNR/SSIM⟦filecite⟧turn4file3⟧
交互提示	SAM2 的 prompt 来源（点/框/自动）	未指定	默认：第一帧人工框选（box）或点选（points）；可扩展：检测器自动出框	SAM2 VideoPredictor API 以“add_new_points_or_box”作为交互入口 [6]
许可约束	各组件许可是否允许课程使用	部分未指定	SAM2：Apache 2.0；ProPainter：S-Lab License 1.0（非商业）；SD-inpainting 权重：CreativeML OpenRAIL-M	SAM2 README 明确 Apache 2.0 [2]；ProPainter LICENSE 明确“非商业用途可再分发/修改，商业需另行许可” [7]；SD-inpainting 模型卡为 OpenRAIL-M [8]
Diffusion 选型	用哪种 Inpainting/是否用 ControlNet	未指定	默认：diffusers `StableDiffusionInpaintPipeline` +（可选）ControlNet-Inpaint	diffusers 官方提供 inpaint pipeline API 及调用签名 [9]；ControlNet 作为条件控制机制可增强结构一致性 [10]
评价指标	质量/速度目标阈值	未指定	质量：JM/JR、PSNR/SSIM +（可选）VFID/LPIPS；性能：端到端时长与峰值 VRAM	课程指定 mask 用 JM/JR、视频用 PSNR/SSIM（有 GT 时）⟦filecite⟧turn4file2⟧；ProPainter repo 提供 VFID 评测依赖（I3D 权重） [3]

备注：为确保计划可执行，建议把所有“未指定项”在你们组内开一次 20 分钟对齐会，冻结成 `configs/project.yaml`，使实验可复现。

目标与交付物

本节把“你要交什么、做到什么程度算完成”写成可验收条款，并直接对齐课程“代码必须公开仓库 + README 可运行 + 必做数据视频打包提交”的硬要求。⟦filecite⟧turn4file4⟧

总体目标：实现一个可一键运行的 Part 3 原型，在 Part 2 的“SAM2 mask + ProPainter inpainting”主链路上，新增 **Diffusion 关键帧/硬区域增强模块**，并完成可视化与定量评估。

交付物清单（可验收）

交付物	形式	验收标准（Definition of Done）
可运行原型（CLI）	`python -m vor.run ...`	在同一台机器上，对 bmx-trees/tennis/Wild Video 可从输入 MP4 输出修复 MP4；日志输出关键配置与耗时；失败时给出可读错误
模块化 Python API	`vor/pipeline.py`	支持 `Pipeline.run(video_path, prompts)->Result`；可单独调用 mask/inpaint/diffusion 子模块
可选 REST 服务	FastAPI	`POST /v1/jobs` 提交任务、`GET /v1/jobs/{id}` 查询、`GET /v1/jobs/{id}/result` 下载结果；GPU 并发限制生效
配置系统	YAML/JSON	所有关键超参可配置（SAM2 checkpoint、ProPainter 参数、Diffusion steps/strength、keyframe 策略等）
测试用例	pytest	至少：mask I/O、ProPainter wrapper、Diffusion wrapper、融合逻辑、端到端小视频（5 帧）
指标评测脚本	`scripts/eval_*.py`	输出 JM/JR、PSNR/SSIM（有 GT 时）；生成对比图/视频（baseline vs +diffusion）
性能基准报告	Markdown 表格	记录：端到端时间、单阶段时间、峰值 VRAM、峰值 RAM；至少覆盖 2 个分辨率档位
可视化材料	`results/figures/*`	至少 3 组：mask overlay、inpaint 对比帧、差分热力图/闪烁对比
仓库 README	文档	写清：环境、依赖、权重下载、运行命令、输出目录结构、复现实验步骤（课程明确要求 README 说明如何运行）filecite turn4file4
videos.zip	产出文件	包含所有 mandatory dataset 的处理后视频（课程要求）filecite turn4file4

模块清单（与交付物对应）

模块	输入	输出	关键依赖/说明
Video I/O	MP4/帧目录	RGB 帧序列、fps、分辨率	推荐 FFmpeg/OpenCV；需保持帧序一致性
Mask 生成（SAM2）	帧序列 + prompts	`T×H×W` 二值 mask（可多目标合并）	SAM2 VideoPredictor：`init_state→add_new_points_or_box→propagate_in_video` [6]
Mask 后处理	mask 序列	平滑后的 mask 序列	膨胀/闭运算、连通域过滤、时序一致性（减少抖动）
ProPainter Inpaint（主干）	帧序列 + mask 序列	inpaint 帧序列	官方脚本 `inference_propainter.py` 支持 mp4/帧 + mask 输入 [3]；提供内存优化参数 [11]

模块	输入	输出	关键依赖/说明
Hard-region/关键帧选择	原帧、mask、ProPainter 输出	hard mask + keyframe idx	对齐“传播不足区域→Diffusion 兜底”的核心思路 filecite turn3file0 filecite turn4file3
Diffusion Inpaint（增强）	keyframes + hard mask	enhanced keyframes	diffusers inpaint pipeline API（prompt/image/mask_image/strength/steps…）[9]；可选 ControlNet 提升结构控制[10]
传播与融合	enhanced keyframes + flow/warp	全序列 refined 输出	关键帧结果用光流或特征传播扩散到邻帧（课程 Direction C 建议）filecite turn4file3
Export	refined 帧序列	MP4、对比视频、日志	课程要求提交处理视频 filecite turn4file4
Eval	GT（可选）	JM/JR、PSNR/SSIM、可视化	课程指定指标 filecite turn4file2 可补充 VFID（ProPainter 提供 I3D 权重位）[3]

详细技术方案

本节给出**可直接按图施工**的架构、数据流、时序流程与接口设计，并明确模型调用顺序为：**SAM2 → ProPainter → Diffusion（关键帧/硬区域）**，严格满足你提出的“以 ProPainter 时序为首要补充、Diffusion 为补充”的要求。filecite turn3file0

核心设计原则（工程约束）

1. **最小侵入集成**：优先用官方脚本/接口跑通，再逐步抽象为 Python API，避免一上来大改 ProPainter/SAM2 源码导致调参与 debug 成本爆炸。ProPainter 官方已提供标准推理脚本和多项“内存友好”参数（neighbor_length/ref_stride/subvideo_length/fp16），可直接用于工程化落地。[12]
2. **Diffusion 只做“少量帧、少量区域”**：把 Diffusion 的使用限制在 hard region / keyframes，符合课程“关键帧生成式修复 + 传播”的建议，同时控制算力与闪烁风险。filecite turn4file3
3. **时序一致性优先于单帧观感**：ProPainter 是主干的原因是其利用时序冗余与传播/Transformer 机制提升一致性；Diffusion 要通过“关键帧 + 传播/融合”而非逐帧独立生成来避免 flicker。[13]

架构图（Mermaid）

```
flowchart LR
  A[Input Video MP4/Frames] --> B[Frame Decoder<br/>RGB frames + fps]
  B --> C[SAM2 Mask Generator<br/>init_state/add_new_points_or_box/propagate_in_video]
  C --> D[Mask Post-process<br/>dilate + temporal smooth]
  D --> E[ProPainter Inpaint (Main)<br/>dual-domain propagation + sparse transformer]
  E --> F[Hard-region & Keyframe Selector<br/>observability/quality cues]
  F -->|keyframes + hard_mask| G[Diffusion Inpaint (Aux)<br/>StableDiffusionInpaint(+ControlNet optional)]
  G --> H[Patch Propagation to Neighbors<br/>optical-flow warp + occlusion check]
```

```
  H --> I[Fusion & Temporal Stabilization<br/>alpha blend + temporal filter]
  I --> J[Video Encoder<br/>MP4 + side-by-side comparisons]
  J --> K[Metrics & Reports<br/>JM/JR, PSNR/SSIM, perf]
```

时序流程（Mermaid Sequence）

```
sequenceDiagram
  participant U as User/Config
  participant IO as VideoIO
  participant M as SAM2Mask
  participant P as ProPainter
  participant S as Selector
  participant D as Diffusion
  participant W as Warp/Fuse
  participant O as Output

  U->>IO: load(video_path)
  IO->>M: frames, prompts
  M->>M: init_state(); add_new_points_or_box(); propagate_in_video()
  M-->>IO: mask_seq
  IO->>P: frames + mask_seq
  P-->>IO: inpaint_seq
  IO->>S: frames + mask_seq + inpaint_seq
  S-->>D: keyframes + hard_mask
  D-->>W: enhanced_keyframes
  W-->>O: refined_seq (all frames)
  O-->>U: mp4 + logs + metrics
```

关键算法与模型调用顺序

阶段一：SAM2 掩码生成（视频多目标分割/跟踪）
SAM2 官方 README 给出标准视频推理 API：`build_sam2_video_predictor` 构建 predictor，`init_state` 初始化视频状态，`add_new_points_or_box` 在某帧加入点/框提示并即时得到 mask，然后 `propagate_in_video` 将该对象 mask “传播”到全视频帧序列。 [6]
工程化要点是把输出标准化为 `mask[t][h][w] ∈ {0,1}`，并把多对象 mask 通过 OR 合并成“需要移除的总 mask”（或保留每个 object_id 以便调试与可解释性）。

阶段二：ProPainter 时序补充（主干 inpainting）
ProPainter 论文与项目页强调其由“增强的传播 + 高效 Transformer”构成，提出 dual-domain propagation（图像域 + 特征域）与 mask-guided sparse video Transformer，以提升大遮挡下的恢复效果与效率。 [14]
实际落地建议直接调用官方 `inference_propainter.py`：它支持输入 mp4 或帧目录、以及逐帧 mask（目录或单张 mask），并可通过 `--neighbor_length`、`--ref_stride`、`--subvideo_length`、`--fp16`、`--width/--height` 等参数进行显存/速度权衡。 [12]

阶段三：Diffusion 增强（关键帧/硬区域局部修复）
课程 Direction C 明确建议：当 ProPainter 因“背景从未出现”而失败时，引入 Stable Diffusion（Inpainting/ControlNet）对关键帧生成式修复，再把结果传播到其他帧。⟦filecite⟧turn4file3⟧
在工程层面，建议使用 Hugging Face [15] diffusers 的 `StableDiffusionInpaintPipeline`（或等价 inpaint pipeline），其 `__call__` 支持 `prompt`、`image`、`mask_image`、`strength`、`num_inference_steps`、

`guidance_scale` 等参数，便于“以 ProPainter 输出为底图、只重绘 hard region”。 [9]
如需增强结构约束，可选用 ControlNet：其论文提出通过额外条件（如边缘、深度、分割等）对扩散模型施加空间控制，减少生成内容偏离场景布局。 [16]
许可方面必须在 README 写清：Stable Diffusion inpainting 权重通常为 CreativeML OpenRAIL-M（带责任使用条款），而 ProPainter 代码/模型为 S-Lab License 1.0（非商业）。 [17]

接口定义（Python API 与 REST API）

Python 侧推荐接口（函数签名）
（用 dataclass 固化配置，保证复现）

```python
from dataclasses import dataclass
from pathlib import Path
from typing import Any, Dict, List, Optional, Tuple
import numpy as np

@dataclass
class Sam2Config:
    model_cfg: str
    checkpoint: str
    device: str = "cuda"
    autocast_dtype: str = "bfloat16"   # or "float16"
    vos_optimized: bool = False         # torch.compile option

@dataclass
class PropainterConfig:
    neighbor_length: int = 10
    ref_stride: int = 10
    subvideo_length: int = 80
    fp16: bool = True
    width: Optional[int] = None
    height: Optional[int] = None
    resize_ratio: float = 1.0

@dataclass
class DiffusionConfig:
    model_id: str = "stable-diffusion-v1-5/stable-diffusion-inpainting"
    num_steps: int = 30
    guidance_scale: float = 7.5
    strength: float = 0.6
    use_controlnet: bool = False
    controlnet_id: Optional[str] = None

@dataclass
class PipelineConfig:
    sam2: Sam2Config
    propainter: PropainterConfig
    diffusion: DiffusionConfig
    keyframe_stride: int = 10           # keyframe every N frames
    hard_region_thresh: float = 0.3   # example
```

```python
class VideoObjectRemovalPipeline:
    def __init__(self, cfg: PipelineConfig):
        ...

    def run(
        self,
        video_path: Path,
        prompts: Dict[str, Any],        # e.g., points/box on first frame
        output_dir: Path,
        gt_dir: Optional[Path] = None
    ) -> Dict[str, Any]:
    """Return paths & metrics."""
        ...
```

REST API（可选，满足“API 设计”要求）
建议最小集合如下，用于批处理/演示：

方法	路径	请求体	返回	说明
POST	`/v1/jobs`	JSON（含 video、prompts、cfg overrides）	`{job_id}`	提交任务
GET	`/v1/jobs/{job_id}`	-	`{status, progress, logs}`	查询进度
GET	`/v1/jobs/{job_id}/result`	-	二进制（zip/mp4）	下载结果
DELETE	`/v1/jobs/{job_id}`	-	`{ok}`	清理缓存

数据格式示例（JSON 与二进制）

任务配置 JSON（示例）
（建议保存到 `runs/<id>/config.json`，保证可追溯）

```json
{
 "video_path": "data/wild/corridor.mp4",
 "prompts": {
  "frame_idx": 0,
  "objects": [
   {"object_id": 1, "box_xyxy": [120, 80, 320, 420]}
  ]
 },
 "sam2": {
  "model_cfg": "configs/sam2.1/sam2.1_hiera_l.yaml",
  "checkpoint": "checkpoints/sam2.1_hiera_large.pt",
  "vos_optimized": true
 },
 "propainter": {
```

```
    "neighbor_length": 10,
    "ref_stride": 10,
    "subvideo_length": 80,
    "fp16": true,
    "width": 576,
    "height": 320
  },
  "diffusion": {
    "model_id": "stable-diffusion-v1-5/stable-diffusion-inpainting",
    "num_steps": 30,
    "strength": 0.6,
    "use_controlnet": false
  },
  "keyframe_stride": 10
}
```

mask 序列存储（建议标准）
- `masks/000000.png ...`：8-bit 单通道 PNG，像素值 0/255。
- 多目标可额外保存 `masks_obj/obj_{id}/000000.png` 便于 debug。
该格式与 ProPainter“frame-wise mask(s)”输入约定对齐，便于直接复用官方脚本。 [3]

并发与资源管理策略

离线脚本模式（课程优先推荐）
- 单机单 GPU：默认 **单视频串行**（避免同时加载 ProPainter + Diffusion 造成 OOM）。
- CPU 上并行：帧解码、mask 后处理、视频编码可用 `ThreadPoolExecutor`；GPU 阶段保持互斥。
- 模型缓存：SAM2、ProPainter、Diffusion pipeline 均应在进程内单例加载，避免频繁加载权重造成 I/O 抖动。

服务模式（可选）
- 每张 GPU 启 1 个 worker（进程级互斥）；任务队列排队。
- 对 Diffusion 做“关键帧批处理”：一次推理多个 keyframes（`num_images_per_prompt` 或把 `image/mask_image` 做 list），尽量提升吞吐，但要监控 VRAM 峰值。diffusers pipeline 明确支持 list 输入。 [9]
- ProPainter 的内存控制：优先调小 `subvideo_length` 与缩放分辨率；官方给出 subvideo_length 与分辨率下的显存估算表，可作为调参上界。 [5]

开发计划与里程碑

为便于执行与验收，推荐按 10 个工作日（两周）拆分；如果你们时间更紧，可合并 Day 6–8，把 Diffusion 仅做“逐帧局部 inpaint（不传播）”作为最小可交版本，再迭代传播。该迭代策略与“先最小可运行版本，再增强”的风险控制思想一致。filecite turn3file0

里程碑时间表（可直接用作甘特依据）

时间	任务	产出	验收标准
Day 1	仓库骨架 + 数据 I/O	`vor/` 包、`scripts/`、`configs/`、视频解码/编码	任意 mp4 可解码成帧并重编码回 mp4，fps 不变

时间	任务	产出	验收标准
Day 2	SAM2 接入（最小闭环）	`Sam2MaskProvider`	给第一帧 box/points，能输出全序列 mask PNG；能 overlay 可视化
Day 3	Mask 后处理与稳定	`mask_post.py`	mask 抖动降低；连通域过滤可配置；输出统计（面积/数量）
Day 4–5	ProPainter wrapper + baseline 跑通	`PropainterRunner` + baseline 结果	能用官方脚本参数跑 bmx-trees/tennis，并保存 results；记录耗时与 VRAM（手工也可） [3]
Day 6	hard region/关键帧策略 v1	`selector.py`	输出 keyframe idx + hard_mask；至少能在“长时遮挡/背景未出现”样例上选出合理区域（可人工检查）filecite turn3file0
Day 7	Diffusion inpaint 接入（关键帧）	`diffusion_inpaint.py`	对选定关键帧输出增强图；支持 strength/steps；记录耗时 [9]
Day 8	关键帧结果传播与融合	`warp_fuse.py`	关键帧增强可影响邻帧；输出无明显接缝（blend）且闪烁可控filecite turn4file3
Day 9	评测与可视化脚本	`eval_mask.py`、`eval_video.py`、`make_figures.py`	输出 JM/JR、PSNR/SSIM（有 GT 时）与对比视频；表格自动生成filecite turn4file2
Day 10	README、Docker、CI	`README.md`、`Dockerfile`、`GH Actions`	新机器按 README 一键复现；Docker 能运行 demo；CI 通过

风险与应对（表格化）

风险	触发信号	影响	优先级	应对措施
依赖冲突（SAM2 需高版本 torch，ProPainter 可能不兼容）	import/算子报错、结果异常	阻塞集成	高	预案 A：单仓库但分两套 env，pipeline 用子进程调用；预案 B：Docker Compose 多服务拆分；预案 C：将 ProPainter 固定在其官方依赖范围，SAM2 输出落盘再调用 [18]
VRAM OOM	ProPainter 或 Diffusion 运行中 OOM	无法跑必做数据	高	ProPainter 调小 `subvideo_length`、降低分辨率、开启 `--fp16`，官方提供显存估算表可做上界控制 [5]；Diffusion 只跑关键帧
mask 边缘不稳导致修复边界抖动	修复边缘“跳动”	质量下降	中	mask 时序平滑 + 形态学；必要时在 ProPainter 前对 mask 做轻微膨胀以覆盖运动模糊（课程亦建议通过 dilation 覆盖模糊边界的思想）filecite turn4file1
Diffusion 逐帧闪烁（flicker）	相邻帧纹理随机变化	观感差	中	只跑关键帧 + 光流传播；对差分做时间滤波；必要时减少 strength、增加对 ProPainter 底图约束filecite turn4file3

风险	触发信号	影响	优先级	应对措施
许可误用	商用/公开发布不合规	合规风险	中	README 标注许可证：SAM2 Apache 2.0、ProPainter 非商业、SD-inpainting OpenRAIL-M；课程作业一般属于非商业使用，但仍需在仓库清晰声明 [19]

测试与评估

课程要求你们在 Experiments 中呈现定量表格与定性可视化，并指定 mask 评测与视频质量评测指标；因此测试体系要覆盖：组件正确性、端到端可复现、指标与可视化自动化。filecite turn4file2

测试分层与用例设计

测试层	目标	样例用例	通过标准
单元测试	函数级正确性	mask I/O、形态学、blend、warp 边界处理	输出 shape/dtype 正确；mask 仍为二值；无越界
集成测试	模块间契约正确	SAM2→mask_dir→ProPainter 输入；ProPainter→Diffusion 输入	完整跑通 5 帧小视频；无文件缺失/排序错误
端到端示例	可复现实验	bmx-trees、tennis、Wild Video 一键跑	输出 mp4 + 对比视频 + 日志 + 配置快照 filecite turn4file4
回归测试	防止改动引入退化	固定 seed + 固定 10 帧片段	指标波动在阈值内；关键帧输出 hash 可比对

定量指标（质量）

- **Mask 质量**：IoU mean (JM) 与 IoU recall (JR) 为课程指定指标。filecite turn4file2
- **Video 质量**：PSNR 与 SSIM（仅在有 GT 的数据上计算）为课程指定指标，且课程提示参考 ProPainter。filecite turn4file2
- **可选补充指标**：
- VFID：ProPainter repo 明确提供 I3D 权重用于评测 VFID，可作为增强项（如果你们要冲高分）。 [3]
- LPIPS：用于衡量感知差异（无 GT 时不适用，或只能做相对比较）。

性能基准（延迟、内存、VRAM）

建议以“阶段耗时 + 峰值资源”方式做表格，至少记录：

- SAM2：mask 生成总耗时 + FPS（可参考 SAM2 repo 也提供 benchmark/速度表格与 torch.compile 优化提示；并支持 `vos_optimized=True`）。 [20]
- ProPainter：端到端耗时、峰值 VRAM；同时记录使用的 `subvideo_length`、分辨率、fp16。ProPainter README 给出参数与显存估算，可作为你们实验设置说明依据。 [5]
- Diffusion：每张关键帧耗时、steps、strength；以及关键帧数量对总耗时的影响。

可视化结果示例（建议你们最终报告中至少包含）

1. `mask_overlay`：原帧 + mask（红色透明叠加）
2. `baseline_vs_hybrid`：ProPainter baseline 与加入 Diffusion 后同帧对比（并标注 hard mask 区域）
3. `temporal_strip`：取同一空间位置的时间条（例如裁剪 ROI 的连续帧拼接），展示 flicker 是否降低
4. `diff_map`：`|I_hybrid - I_base|` 热力图（显示 Diffusion 实际改变了哪些区域）

部署与运行

本节目标是让你的仓库满足课程“代码可运行、依赖清晰、结果可复现”的评分点，并提供本地与云端两套可选部署方式。filecite turn4file4

依赖清单（建议拆分三层）

1. **基础**：Python 3.10+、FFmpeg、numpy、opencv-python、Pillow、tqdm、pyyaml、pytest
2. **深度学习**：torch/torchvision（按 SAM2 要求），以及 ProPainter 依赖（从其 requirements.txt 抽取并锁版本） [18]
3. **Diffusion**：diffusers、transformers、accelerate、safetensors（以及可选 ControlNet 权重）

权重管理策略（必须写进 README）

- SAM2：官方提供 checkpoints 与 config；README 中给出安装/加载方式，并说明其 Apache 2.0 许可。 [20]
- ProPainter：README 指引从 Releases 下载到 `weights/`，也可首次推理自动下载；并列出 `ProPainter.pth`、`recurrent_flow_completion.pth`、`raft-things.pth` 等文件。 [3]
- Diffusion：使用 diffusers 从模型仓库拉取（注意遵守 OpenRAIL-M）。 [21]

Dockerfile 示例（单容器版，适合离线脚本）

```
FROM nvidia/cuda:12.4.1-cudnn-runtime-ubuntu22.04

RUN apt-get update && apt-get install -y \
  python3 python3-pip git ffmpeg && \
  rm -rf /var/lib/apt/lists/*

WORKDIR /app
COPY . /app

# Install torch/torchvision first (pin in requirements.lock in repo)
RUN pip3 install --no-cache-dir -r requirements.txt

# Install SAM2 as editable (if vendored as submodule) and other deps
# RUN pip3 install -e third_party/sam2

ENV PYTHONUNBUFFERED=1
CMD ["python3", "-m", "vor.run", "--help"]
```

docker-compose 示例（多服务版，解决依赖冲突预案）
（当 SAM2 与 ProPainter 需要不同 torch 版本时，用文件/HTTP 作为阶段间接口）

```yaml
services:
  sam2_mask:
    build: ./services/sam2
    runtime: nvidia
    volumes: ["./shared:/shared"]
  propainter:
    build: ./services/propainter
    runtime: nvidia
    volumes: ["./shared:/shared"]
  diffusion:
    build: ./services/diffusion
    runtime: nvidia
    volumes: ["./shared:/shared"]
```

CI/CD（建议适度即可）
课程不要求 CI/CD，但它能显著减少“环境跑不起来”的风险。建议 GitHub [1] Actions 做三件事：

1. `lint`：ruff/black（可选）
2. `unit tests`：pytest（不跑 GPU 测试，只跑纯 CPU 的 mask/融合逻辑）
3. `build`：Docker build（可选，确保 Dockerfile 不坏）

参考与来源优先级

本计划书的来源优先级按“官方仓库/论文/文档 > 官方模型卡与许可证 > 高质量中文资料 > 非官方教程”执行，并且与课程要求“必须阅读并引用推荐论文、并在报告中给出技术路线与可视化”一致。【filecite】turn4file4【

一级（最高优先级：官方仓库与官方文档）
- SAM2 官方仓库：安装要求、VideoPredictor API、许可证（Apache 2.0）。 [20]
- ProPainter 官方仓库：推理脚本、输入输出约定、显存优化参数与显存估算表、权重目录。 [22]

二级（论文/开放获取 PDF）
- ProPainter 论文：dual-domain propagation 与高效 Transformer 的方法依据。 [23]
- SAM2 论文/开放评审版本：模型定位与视频分割设定（promptable video segmentation）。 [24]
- Latent Diffusion（Stable Diffusion 的 LDM 基础）：LDM 在更低维潜空间扩散以降低计算/内存，且适用于 inpainting 等任务。 [25]
- ControlNet 论文：为扩散模型引入空间条件控制的架构依据，有助于增强结构一致性。 [26]

三级（模型卡/许可证与工程 API 文档）
- diffusers inpaint pipeline 文档与调用签名。 [27]
- Stable Diffusion inpainting 权重许可（CreativeML OpenRAIL-M）。 [28]
- ProPainter 非商业许可证条款（S-Lab License 1.0）。 [7]

四级（中文资料优先：仅作辅助理解/快速查阅）
- 中文模型介绍/镜像页面可用于快速确认许可证与模型定位，但工程与学术表述以官方为准。 [29]

与你当前“上面的内容与相关文档”对齐的内部材料
- 你前序的 Part 3 混合路线与 hard region 思路文档：【filecite】turn0file0【（同一内容亦见【filecite】turn3file0【

- 课程 Project 3 任务说明与 Part 3 Direction C 关键帧 diffusion 建议、指标与交付要求：filecite turn0file1（同一内容亦见 filecite turn4file3 filecite turn4file2 filecite turn4file4

---

1 13 14 23 https://arxiv.org/abs/2309.03897
https://arxiv.org/abs/2309.03897

2 4 6 18 19 20 https://github.com/facebookresearch/sam2
https://github.com/facebookresearch/sam2

3 5 11 12 15 22 https://github.com/sczhou/ProPainter
https://github.com/sczhou/ProPainter

7 https://github.com/sczhou/ProPainter/blob/main/LICENSE
https://github.com/sczhou/ProPainter/blob/main/LICENSE

8 17 21 28 https://huggingface.co/stable-diffusion-v1-5/stable-diffusion-inpainting
https://huggingface.co/stable-diffusion-v1-5/stable-diffusion-inpainting

9 27 https://huggingface.co/docs/diffusers/api/pipelines/stable_diffusion/inpaint
https://huggingface.co/docs/diffusers/api/pipelines/stable_diffusion/inpaint

10 16 26 https://openaccess.thecvf.com/content/ICCV2023/papers/Zhang_Adding_Conditional_Control_to_Text-to-Image_Diffusion_Models_ICCV_2023_paper.pdf
https://openaccess.thecvf.com/content/ICCV2023/papers/Zhang_Adding_Conditional_Control_to_Text-to-Image_Diffusion_Models_ICCV_2023_paper.pdf

24 https://arxiv.org/html/2408.00714v2
https://arxiv.org/html/2408.00714v2

25 https://arxiv.org/abs/2112.10752
https://arxiv.org/abs/2112.10752

29 https://www.atyun.com/models/info/runwayml/stable-diffusion-inpainting.html
https://www.atyun.com/models/info/runwayml/stable-diffusion-inpainting.html